Software Requirement Specification (SRS) for DBMS Project

1. Introduction

1.1 Purpose

The purpose of the **AI-Based Remote Server Monitoring Database System (Sentinel AI)** is to design and implement a centralized software platform that **monitors, stores, and analyzes server performance data in near real time**. The system enables administrators to track critical server parameters such as **CPU usage, memory utilization, disk usage, temperature, and system status** using a lightweight Python-based monitoring agent.

The collected data is transmitted to a centralized backend and stored in a **relational database** for efficient retrieval and analysis. An **AI-based anomaly detection module using Isolation Forest** identifies abnormal behavior in server performance and generates alerts. The database ensures **secure storage, structured organization, and fast access** to large volumes of monitoring data.

1.2 Scope

The **Remote Server Monitoring Database System** is a **web-based application** developed to monitor and manage multiple servers from a unified dashboard. The system uses a **Flask-based backend API** and a **React-based frontend interface** to visualize server health, performance metrics, and alerts.

Server performance data is collected using a **Python monitoring agent** and transmitted to the backend via **HTTP APIs**. The backend processes and stores this data in an **RDBMS (SQLite / MySQL / PostgreSQL)**. The system automates **monitoring, alert generation, data storage, and visualization**, ensuring scalable and reliable server supervision.

The platform supports **role-based access** for different users and provides a foundation for **AI-driven anomaly detection**, making it suitable for **data centers, enterprise IT systems, and cloud infrastructures**.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
DBMS	Database Management System
RDBMS	Relational DBMS using tables (MySQL)
MySQL	Open-source relational database
IoT	Internet of Things
MQTT	Message Queuing Telemetry Transport protocol
HTTP	Hypertext Transfer Protocol
AI	Artificial Intelligence
ML	Machine Learning

Term	Definition
LSTM	Algorithm for time-series prediction
CRUD	Create, Read, Update, Delete
ERD	Entity Relationship Diagram
UI	User Interface
UX	User Experience
API	Application Programming Interface
CPU	Central Processing Unit

2. Overall Description

2.1 Product Perspective

The system follows a **three-tier architecture**:

- **Presentation Layer** – React-based web dashboard
- **Application Layer** – Flask backend with REST APIs
- **Database Layer** – Relational database (RDBMS)

A Python-based monitoring agent installed on each server collects performance metrics and sends them to the backend using HTTP requests. The backend stores the data in the database and triggers AI-based anomaly detection. The frontend dashboard visualizes server status, metrics, and alerts.

The system is scalable and can be deployed in **local, on-premise, or cloud environments**.

2.2 Product Functions

User Authentication

- Secure login for users
- Role-based access control
- Encrypted password storage

Data Collection & Monitoring

- Automatic collection of CPU, RAM, disk, temperature data.
- Periodic metric updates from servers

Server Management

- Admin registers servers.
- Logs and metrics stored automatically.

Alerts & Notifications

- Automatic alert generation on anomaly detection.
- Notification for overload and failures.

AI-Based Intelligence

- Anomaly detection.
- Failure prediction.

Data Storage and Reporting

- Time-series metric storage
- Alert history and logs

2.3 User Characteristics

Admin: Full control over servers, database, and users.

Operator: Monitors performance and responds to alerts.

Viewer: Read-only access to dashboard

2.4 Constraints

Hardware:

- Processor: Intel i3 or higher
- RAM: Minimum 4GB
- Disk: 50GB free
- Display: 1024×768 minimum

Software:

- Backend: Python + Flask
- Frontend: React (Vite) with TypeScript
- Database: SQLite
- Protocols: HTTP
- OS: Windows/Linux/Mac
- Browser: Chrome/Firefox/Edge

Database:

- MySQL design using 3NF
- Backup system
- Designed for concurrent access

2.5 Assumptions and Dependencies

Assumptions:

- Admin has technical knowledge.
- Data is accurate.
- Agents remain active.
- Network connectivity is stable.

Dependencies:

- Python 3.x
- Flask
- React & Node.js
- Relational Database Engine

3. Specific Requirements

3.1 Functional Requirements

Module	ID	Description
Authentication	FR-1	User login system
	FR-2	Role access
	FR-3	Admin manages users
Server Mgmt	FR-4	Admin registers servers
Monitoring	FR-5	Collect metrics
	FR-6	View dashboard
Logs	FR-7	Store logs
Alerts	FR-8	Auto alerts
AI	FR-9	Anomaly detection
	FR-10	Failure prediction
Reports	FR-11	Generate reports
Security	FR-12	Data encryption

3.2 Non-Functional Requirements

- **Performance:** Supports up to 50 servers with low latency.
- **Reliability:**99% uptime with backups.
- **Security:** Encryption, RBAC, SQL protection.
- **Usability:** Simple UI.
- **Maintainability:** Clean code design.
- **Scalability:** Support more servers.

3.3 AI and ML Requirements

- **Anomaly Detection:** Isolation Forest detects unusual usage.
- **Failure Prediction:** LSTM forecasts future failures.
- **Intelligent Alerts:** AI warnings.
- **Smart Dashboard:** Health score and risk index.